Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Процессы и профессиональная практика программной инженерии»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Алешов Е.В.

ПРАКТИЧЕСКАЯ РАБОТА 6
«Методология MSF»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной практической работы является изучение методологии Microsoft Solutions Framework (MSF), её основных принципов, модели жизненного цикла проекта, особенностей организации команды разработчиков и распределения ответственности между ролевыми кластерами. В работе рассматриваются этапы создания и внедрения ИТ-решения, основные вехи проекта, а также возможности применения MSF для повышения качества разработки программного обеспечения и удовлетворения бизнес-потребностей заказчика.

1.2. Задачи

- Изучить историю развития методологии MSF и особенности её версий.
- Рассмотреть понятие ИТ-решения в методологии Microsoft.
- Изучить основные принципы MSF.
- Объяснить принципы единого видения проекта, гибкости, ориентации на бизнес-приоритеты и свободного обмена информацией.
- Рассмотреть модель жизненного цикла MSF и её отличие от каскадной и спиральной моделей.
- Изучить пять фаз проекта: выработку концепции, планирование, разработку, стабилизацию и внедрение.
- Разобрать назначение главных и промежуточных вех проекта.
- Изучить модель команды MSF и особенности организации команды равных.
- Рассмотреть семь ролевых кластеров и распределение ответственности между ними.
- Изучить принципы масштабирования команды и формирования межфункциональных групп.
- Рассмотреть пример применения MSF при разработке информационной системы.
- Сформулировать выводы о преимуществах и особенностях использования MSF.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. История развития методологии MSF

Методология Microsoft Solutions Framework (MSF) была разработана компанией Microsoft в 1990-х годах на основе практического опыта создания, внедрения и сопровождения заказных ИТ-решений. Её целью было повышение эффективности проектной деятельности, улучшение взаимодействия с заказчиками и субподрядчиками, а также обеспечение качества программных продуктов на всех этапах жизненного цикла. В 2002 году появилась версия MSF 3.1, включавшая руководства по модели процессов, модели команды, управлению проектами, управлению рисками и управлению подготовкой. В 2005 году была представлена версия MSF 4.0, связанная с Visual Studio Team System. Она развивалась по двум направлениям: MSF for Agile, ориентированному на гибкие методы разработки, и MSF for CMMI, предназначенному для более формализованного управления процессами. Версия MSF 4.2 продолжила развитие данного подхода. В отличие от MSF 3.x, ориентированной преимущественно на общие принципы разработки заказных ИТ-решений, версии 4.x предлагали формализованные шаблоны процессов, роли, действия, артефакты и настройки для инструментальной среды Microsoft.

2.2. Понятие ИТ-решения в MSF

ИТ-решение в методологии MSF представляет собой согласованный набор компонентов, необходимых для удовлетворения бизнес-потребностей конкретного заказчика. Оно включает не только программное обеспечение, но и техническую и пользовательскую документацию, обучение персонала, процедуры внедрения, эксплуатацию, поддержку и сопровождение. Поэтому успешность проекта определяется не только созданием работоспособного программного продукта, но и его полноценным внедрением в деятельность организации. Даже качественно разработанная система не принесёт ожидаемой пользы, если пользователи не умеют с ней работать или отсутствуют необходимые процедуры технической поддержки.

2.3. Основные принципы MSF

- Единое видение проекта. Все участники проекта и заказчик должны одинаково понимать цели, ожидаемые результаты, ограничения и критерии успеха. Это позволяет согласовать действия команды и избежать разногласий при принятии решений.
- Гибкость и готовность к переменам. Требования и условия реализации проекта могут изменяться. Команда должна своевременно реагировать на изменения, корректировать планы и документацию, сохраняя ориентацию на конечный результат.
- Концентрация на бизнес-приоритетах. Проект должен обеспечивать реальную пользу заказчику и пользователям. При выборе решений учитываются бизнес-цели, стоимость, сроки, риски и ожидаемая отдача.
- Поощрение свободного общения. Открытый обмен информацией между участниками проекта позволяет своевременно выявлять проблемы, уменьшать неопределённость, предотвращать ошибки и согласовывать действия.
- Качество и ответственность. Каждый участник должен понимать свои обязанности, выполнять принятые обязательства и своевременно сообщать о рисках, которые могут помешать достижению результата.
- Итеративная разработка. Создание решения выполняется последовательными итерациями, позволяющими постепенно расширять функциональность, проверять результаты и учитывать обратную связь.
- Интеграция разработки и внедрения. Проект рассматривается как полный цикл создания и передачи решения в эксплуатацию, включая подготовку пользователей и команды сопровождения.

2.4. Принцип единого видения проекта

Единое видение предполагает наличие общего и согласованного представления о том, какой результат должен быть получен в ходе проекта. Заказчик может ожидать одни функции, разработчики — другие, а пользователи — третьи. Если эти представления не согласованы, возрастает вероятность появления лишней функциональности, нарушения сроков и неудовлетворённости результатом. Для предотвращения подобных ситуаций на начальном этапе формируются описание видения проекта и документ определения его рамок. В них фиксируются цели, ожидаемые возможности, ограничения, основные требования и критерии успешного завершения.

2.5. Гибкость и готовность к переменам

MSF исходит из того, что требования к программному продукту могут изменяться в процессе разработки. Причинами изменений могут быть новые бизнес-задачи, замечания пользователей, технические ограничения и результаты тестирования. Вместо жёсткой фиксации всех решений на начальном этапе методология предполагает регулярный пересмотр требований, планов и проектной документации. При этом изменения должны оцениваться с точки зрения их влияния на сроки, стоимость, риски и качество продукта.

2.6. Концентрация на бизнес-приоритетах

Разработка программного обеспечения должна быть направлена на получение измеримой или обоснованной пользы. Для организации это может выражаться в снижении расходов, ускорении бизнес-процессов, повышении производительности и улучшении качества обслуживания. При ограниченных ресурсах команда должна отдавать предпочтение функциональности, которая обеспечивает наибольшую ценность для заказчика. Важным условием достижения бизнес-результата является внедрение системы, поскольку до начала её реального использования ожидаемая отдача обычно не реализуется в полной мере.

2.7. Поощрение свободного общения

Открытое общение предусматривает своевременное предоставление участникам проекта достоверной информации о ходе работ, выявленных проблемах, изменениях требований и возможных рисках. Команда регулярно оценивает прогресс, фиксирует результаты и обсуждает отклонения от плана. Такая организация взаимодействия позволяет обнаруживать проблемы на ранних этапах, быстрее принимать решения и поддерживать доверие между заказчиком, разработчиками, тестировщиками и специалистами по внедрению.

2.8. Модель жизненного цикла MSF

Модель жизненного цикла MSF сочетает элементы каскадного и спирального подходов. От каскадной модели она использует разделение проекта на фазы с определёнными результатами и контрольными точками. От спиральной модели заимствованы итеративность, оценка рисков, постепенное развитие решения и возможность корректировать требования. Проект проходит пять основных фаз: выработка концепции, планирование, разработка, стабилизация и внедрение. При этом документация и планы могут обновляться по мере получения новых результатов. Разработка начинается с базовой функциональности, после чего система постепенно дополняется новыми возможностями.

2.9. Фазы проекта MSF

- Выработка концепции. Определяются цели и рамки проекта, формируется основная команда, анализируются бизнес-требования и ключевые риски. Основным результатом является согласованное видение решения.
- Планирование. Уточняются требования, создаётся функциональная спецификация, разрабатываются архитектура, планы тестирования, внедрения, эксплуатации и обучения. Оцениваются сроки, ресурсы, стоимость и риски. Результатом становятся согласованные планы проекта.
- Разработка. Создаются программный код и остальные компоненты решения, подготавливаются установочные сценарии, тестовые материалы и документация. Реализованная функциональность регулярно интегрируется и проверяется.
- Стабилизация. Выполняются комплексное тестирование, устранение ошибок, проверка производительности и испытания в условиях, приближённых к реальной эксплуатации. При необходимости проводится пилотное внедрение. Цель фазы — подтвердить готовность продукта к выпуску.
- Внедрение. Решение устанавливается в рабочей среде, настраивается, передаётся пользователям и специалистам сопровождения. Проверяется работоспособность системы, оформляется завершение проекта и оценивается удовлетворённость заказчика.

2.10. Главные и промежуточные вехи

Вехи проекта представляют собой контрольные точки, в которых оцениваются достигнутые результаты и принимаются решения о дальнейшем выполнении работ. Главные вехи обозначают завершение фаз и переход к следующему этапу. В классической модели MSF к ним относятся утверждение концепции, утверждение плана проекта, завершение разработки, утверждение готовности к выпуску и завершение внедрения. Промежуточные вехи фиксируют результаты внутри отдельной фазы: завершение проектирования интерфейса, интеграцию компонентов, окончание тестирования отдельных модулей или устранение критических ошибок. Промежуточные вехи помогают разделить крупные задачи на небольшие контролируемые этапы.

2.11. Модель команды MSF

Модель команды MSF основана на принципе равноправного взаимодействия специалистов, обладающих различными компетенциями и несущих ответственность за общую цель. Такая команда не обязательно имеет единственного формального руководителя, который принимает все решения. Вместо этого ответственность распределяется между ролевыми кластерами, каждый из которых представляет определённую перспективу проекта. Модель предполагает чёткое распределение обязанностей, согласование решений, открытые коммуникации и готовность участников выполнять принятые обязательства. При этом на практике допускается назначение руководителя или изменение структуры ответственности в зависимости от масштаба и особенностей проекта.

2.12. Ролевые кластеры MSF

- Управление продуктом. Представляет интересы заказчика, определяет бизнес-цели и следит за тем, чтобы результат удовлетворял потребности пользователей. Отвечает за согласование требований и ожидаемой ценности продукта.
- Управление программой. Организует выполнение проекта, координирует планы, сроки, ресурсы и бюджет, отслеживает прогресс и помогает устранять препятствия, мешающие работе команды.
- Разработка. Отвечает за реализацию программного обеспечения, написание кода, интеграцию компонентов и исправление дефектов, обнаруженных в процессе разработки.
- Тестирование. Проверяет соответствие продукта требованиям, выявляет дефекты, оценивает надёжность и помогает подтвердить готовность решения к выпуску. Для обеспечения объективности тестирование должно сохранять независимость от разработки.
- Удовлетворение потребителя. Отвечает за удобство использования решения, пользовательский интерфейс, пользовательскую документацию и обучение пользователей.
- Управление выпуском. Организует внедрение и передачу решения в эксплуатацию, обеспечивает подготовку рабочей среды, координацию развёртывания и взаимодействие с командой сопровождения.
- Архитектура. Определяет структуру решения, ключевые технологии, взаимодействие компонентов и основные проектные ограничения. В версии MSF 4.x архитектурная ответственность выделена в отдельный ролевой кластер; в более ранней модели соответствующие обязанности относились преимущественно к управлению программой.

2.13. Масштабирование команды MSF

Количество ролевых кластеров не определяет точное количество сотрудников. В небольшом проекте один специалист может выполнять несколько ролей, если это не приводит к конфликту интересов и снижению качества. Например, разработчик может участвовать в подготовке пользовательской документации, но совмещение разработки и независимого тестирования нежелательно, поскольку разработчик может не заметить собственные ошибки. В крупных проектах команда разделяется на небольшие межфункциональные группы, каждая из которых отвечает за определённую часть продукта. Такие группы включают представителей нескольких ролевых кластеров и регулярно согласуют результаты своей работы. При необходимости внутри отдельных кластеров создаются функциональные группы по направлениям компетенции.

2.14. Управление рисками в MSF

Управление рисками представляет собой непрерывный процесс выявления, анализа, оценки и снижения факторов, способных отрицательно повлиять на проект. Риски могут быть связаны с технической сложностью, недостаточной определённостью требований, нехваткой ресурсов, изменением сроков, вопросами безопасности и трудностями внедрения. На начальном этапе формируется перечень основных рисков, затем он регулярно пересматривается по мере развития проекта. Для значимых рисков определяются вероятность возникновения, возможные последствия, ответственные участники и меры реагирования. Такой подход позволяет своевременно принимать решения и уменьшать вероятность срыва проекта.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Анализ требований заказчика

В качестве примера рассматривается разработка информационной системы учёта заявок для небольшой организации. Система должна позволять сотрудникам создавать заявки, изменять их состояние, назначать исполнителей, отслеживать сроки выполнения и формировать отчёты. Перед началом разработки необходимо согласовать бизнес-цели, определить категории пользователей, описать основные сценарии использования и установить критерии успешности. Такой подход соответствует принципу единого видения MSF и позволяет избежать разработки функций, не представляющих существенной ценности для заказчика.

3.2. Определение рамок проекта

В рамки первой версии системы включаются регистрация заявок, просмотр списка заявок, изменение статусов, назначение ответственных и формирование базового отчёта. Интеграция с внешними информационными системами, автоматические уведомления и расширенная аналитика могут быть перенесены в последующие итерации. Такое ограничение объёма позволяет сосредоточиться на наиболее важных потребностях и уменьшает риск превышения сроков и бюджета.

3.3. Планирование разработки

- Составить функциональную спецификацию системы.
- Определить роли пользователей и основные сценарии использования.
- Подготовить концептуальный, логический и физический дизайн.
- Определить архитектуру приложения и структуру базы данных.
- Составить календарный график, оценить трудозатраты и распределить обязанности.
- Разработать планы тестирования, внедрения, обучения и сопровождения.
- Сформировать реестр рисков и определить меры реагирования.

3.4. Итеративная реализация

Разработка начинается с реализации минимально необходимой функциональности: создания и просмотра заявок. После проверки этой версии добавляются изменение статусов, назначение исполнителей и формирование отчётов. После каждой итерации команда анализирует результаты, получает обратную связь от заказчика, исправляет обнаруженные дефекты и обновляет документацию. Такой порядок позволяет раньше проверить правильность выбранного решения и скорректировать требования до завершения всего проекта.

3.5. Организация тестирования

Тестирование проводится на нескольких уровнях. Модульные тесты проверяют отдельные функции, интеграционные тесты — взаимодействие компонентов, а системные тесты — работу приложения в целом. Дополнительно выполняется проверка пользовательских сценариев и требований к удобству использования. В фазе стабилизации система испытывается в условиях, приближённых к рабочей среде. Найденные дефекты регистрируются, классифицируются по критичности и устраняются до принятия решения о готовности к выпуску.

3.6. Подготовка внедрения и сопровождения

До передачи системы в эксплуатацию подготавливаются инструкции, материалы обучения, процедуры установки и настройки, правила резервного копирования и порядок обработки обращений пользователей. Команда управления выпуском координирует развёртывание, а специалисты разработки и тестирования участвуют в устранении проблем, выявленных после установки. Ответственность за эксплуатацию передаётся назначенным сотрудникам. Такой подход обеспечивает не только техническую готовность продукта, но и способность организации использовать его в повседневной работе.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

4.1. Исходная постановка задачи

Для практического применения MSF рассматривается проект создания информационной системы учёта заявок организации. Заказчик заинтересован в сокращении времени обработки обращений, повышении прозрачности работы сотрудников и получении достоверных сведений о состоянии заявок. До начала реализации необходимо определить требования, согласовать ожидаемые результаты и установить границы первой версии продукта. Главным критерием успеха является создание работающей системы, которая удовлетворяет согласованным требованиям и может использоваться в реальной деятельности.

4.2. Выработка концепции проекта

- Определена бизнес-проблема: заявки обрабатываются без единого централизованного учёта.
- Сформулирована цель: создать систему регистрации, распределения и контроля заявок.
- Определены основные пользователи: сотрудники, исполнители и администратор.
- Согласованы первоначальные рамки: создание заявок, просмотр, изменение статусов и базовая отчётность.
- Выявлены предварительные риски: изменение требований, задержки разработки, ошибки при хранении данных и недостаточная подготовка пользователей.
- Сформировано единое видение ожидаемого результата между заказчиком и проектной командой.

4.3. Планирование проекта

- Подготовлена функциональная спецификация, описывающая поведение системы.
- Определены пользовательские сценарии: создание заявки, назначение исполнителя, изменение статуса и просмотр отчёта.
- Спроектирована общая архитектура приложения и структура хранения данных.
- Разработан предварительный календарный график работ.
- Распределены обязанности между разработчиками, тестировщиками, специалистами по пользовательскому взаимодействию и внедрению.
- Предусмотрены тестирование, обучение сотрудников, подготовка документации и передача системы на сопровождение.

4.4. Разработка решения

На фазе разработки реализуется основная функциональность системы. Сначала создаётся базовая версия, позволяющая регистрировать и просматривать заявки. Затем добавляются изменение статусов, назначение исполнителей и формирование отчётов. Программный код регулярно проверяется и интегрируется. По мере появления новых результатов обновляются проектная документация и планы. Последовательная реализация позволяет получить работоспособную основу системы до завершения всех дополнительных функций.

4.5. Стабилизация решения

После реализации основных функций проводится комплексное тестирование. Проверяется корректность создания заявок, сохранения данных, изменения статусов, назначения исполнителей и формирования отчётов. Дополнительно оцениваются обработка ошибочного ввода, доступ пользователей к разрешённым функциям и устойчивость системы при типовой нагрузке. Все найденные дефекты регистрируются и исправляются. При необходимости проводится пилотное испытание с участием ограниченной группы сотрудников. По результатам проверки принимается решение о готовности системы к внедрению.

4.6. Внедрение системы

- Система устанавливается и настраивается в рабочей среде организации.
- Проверяется доступность приложения и корректность работы основных функций.
- Пользователям предоставляются инструкции и проводится обучение.
- Назначаются ответственные сотрудники за эксплуатацию и поддержку.
- Определяются процедуры регистрации обращений, устранения ошибок и резервного копирования.
- Заказчик проверяет соответствие результата согласованным требованиям и подтверждает принятие системы.

4.7. Применение принципа единого видения

В рассматриваемом проекте единое видение достигается согласованием перечня функций и критериев приёмки до начала основной разработки. Например, заказчик должен подтвердить, какие статусы заявки необходимы, кто вправе назначать исполнителей и какие данные должны отображаться в отчёте. Если такие вопросы оставить нерешёнными, команда может реализовать функциональность, которая не соответствует ожиданиям пользователей. Документирование согласованных требований снижает вероятность разногласий и помогает объективно оценивать готовность результата.

4.8. Применение принципа гибкости

Предположим, после демонстрации первой версии заказчик просит добавить фильтрацию заявок по исполнителю. В рамках MSF запрос анализируется, оценивается по трудозатратам и влиянию на сроки, после чего команда принимает решение о включении функции в текущую или следующую итерацию. При необходимости обновляются спецификация и календарный график. Таким образом, изменение требований не рассматривается как исключительная ситуация, но и не принимается без оценки его последствий.

4.9. Применение принципа бизнес-приоритетов

В первую очередь реализуются функции, без которых невозможно организовать централизованный учёт заявок: регистрация, просмотр и изменение статусов. Дополнительные возможности, например расширенная аналитика или интеграция с внешними сервисами, могут быть отложены. Такое распределение приоритетов позволяет быстрее предоставить пользователям полезный продукт и уменьшает риск того, что проект завершится после расходования ресурсов без получения работающего решения.

4.10. Применение принципа свободного общения

Для поддержания открытого обмена информацией команда регулярно проводит рабочие встречи, демонстрирует промежуточные результаты и фиксирует обнаруженные проблемы. Заказчик своевременно получает сведения о выполненных задачах, рисках и возможных изменениях сроков. Тестировщики сообщают разработчикам о дефектах, а специалисты по внедрению заранее информируют команду о требованиях рабочей среды. Такой порядок помогает обнаруживать проблемы до завершения проекта и сокращает расходы на их исправление.

4.11. Распределение ответственности по ролевым кластерам

- Управление продуктом согласовывает требования с заказчиком и определяет приоритеты функциональности.
- Управление программой составляет график, координирует работу участников, отслеживает сроки и риски.
- Разработка реализует функции приложения, структуру программных компонентов и взаимодействие с базой данных.
- Тестирование проверяет выполнение требований, обнаруживает дефекты и подтверждает качество системы.
- Удовлетворение потребителя разрабатывает понятный интерфейс, пользовательские инструкции и материалы обучения.
- Управление выпуском готовит рабочую среду, организует установку системы и передачу её в эксплуатацию.
- Архитектура определяет общую структуру приложения, взаимодействие компонентов и ключевые технические решения.

4.12. Анализ взаимодействия ролевых кластеров

Ролевые кластеры не работают изолированно друг от друга. Управление продуктом определяет требования и приоритеты, архитектура и разработка преобразуют их в техническое решение, тестирование проверяет соответствие требованиям, а специалисты по удовлетворению потребителя оценивают удобство использования. Управление программой координирует выполнение задач, а управление выпуском готовит внедрение и последующее сопровождение. Например, при изменении формы заявки необходимо согласовать бизнес-потребность, обновить архитектуру и программную реализацию, проверить новую форму, скорректировать документацию и подготовить пользователей к изменениям. Согласованная работа всех кластеров обеспечивает целостность ИТ-решения.

4.13. Главные вехи рассматриваемого проекта

- Веха 1 — концепция утверждена: согласованы бизнес-цели, видение и рамки проекта.
- Веха 2 — план проекта утверждён: подготовлены требования, спецификация, график и основные планы работ.
- Веха 3 — разработка завершена: реализована согласованная функциональность и подготовлены необходимые материалы.
- Веха 4 — готовность к выпуску утверждена: завершены основные испытания, критические дефекты устранены, продукт признан готовым к внедрению.
- Веха 5 — внедрение выполнено: система работает в реальной среде, пользователи обучены, а ответственность за сопровождение передана.

4.14. Оценка рисков проекта

- Изменение требований. Снижается путём согласования рамок проекта, регулярного обсуждения запросов и оценки их влияния на график.
- Превышение сроков. Снижается путём декомпозиции задач, регулярного контроля прогресса и определения приоритетов.
- Ошибки программного обеспечения. Снижаются посредством код-ревью, автоматизированных и ручных тестов, а также исправления дефектов до выпуска.
- Недостаточная готовность пользователей. Снижается путём обучения, подготовки документации и проведения пилотного внедрения.
- Проблемы эксплуатации. Снижаются подготовкой инструкций, процедур резервного копирования, мониторинга и технической поддержки.

4.15. Сравнение MSF с каскадной и спиральной моделями

Каскадная модель предполагает последовательное прохождение этапов и обычно требует подробного определения требований до начала основной разработки. Спиральная модель основывается на повторяющихся циклах разработки, оценке рисков и постепенном уточнении решения. MSF объединяет отдельные особенности обоих подходов: использует чёткие фазы и контрольные вехи, но допускает итеративную разработку, обратную связь и изменение планов. В отличие от строго последовательного процесса, в MSF тестирование, подготовка внедрения и сопровождения учитываются на протяжении проекта, а не только после завершения программирования.

4.16. Преимущества и ограничения MSF

- Преимущества: согласование ожиданий заказчика и команды, распределение ответственности, регулярная оценка рисков, адаптация к изменениям, постепенная проверка функциональности и внимание к внедрению.
- Преимущества: возможность масштабировать команду под размер проекта и объединять несколько компетенций в межфункциональных группах.
- Ограничения: необходимость дисциплинированного взаимодействия, документирования решений и регулярного согласования изменений.
- Ограничения: модель равноправной команды требует достаточной профессиональной зрелости участников и не отменяет необходимости разрешать конфликты и принимать окончательные решения.
- Ограничения: формальные шаблоны MSF 4.x связаны с инструментальной средой Visual Studio Team System, поэтому их использование следует отличать от общих принципов более ранних версий MSF.

4.17. Оценка результата практического применения

В рассмотренном примере методология MSF позволяет организовать создание системы учёта заявок как последовательность согласованных фаз с проверяемыми результатами. На этапе концепции определяется общая цель, при планировании формируются требования и график, при разработке создаётся функциональность, при стабилизации подтверждается качество, а при внедрении система передаётся пользователям и специалистам сопровождения. Результат оценивается по соответствию согласованным требованиям, работоспособности основных функций, устранению критических ошибок, готовности пользователей и наличию процедур поддержки. Числовые показатели трудозатрат и производительности в данном примере не измерялись, поскольку практическое выполнение представляет собой аналитическую модель применения методологии, а не эксперимент с реальной программной системой.

V. ВЫВОДЫ

В ходе практической работы была изучена методология Microsoft Solutions Framework (MSF), рассмотрены история её развития, основные принципы, модель жизненного цикла и особенности организации проектной команды. Установлено, что MSF рассматривает ИТ-решение как комплекс программного обеспечения, документации, обучения, внедрения и сопровождения, направленный на удовлетворение бизнес-потребностей заказчика. Изучены четыре основных принципа: единое видение проекта, гибкость и готовность к переменам, концентрация на бизнес-приоритетах и свободный обмен информацией. Рассмотрены пять фаз проекта — выработка концепции, планирование, разработка, стабилизация и внедрение — а также назначение главных и промежуточных вех. Изучены семь ролевых кластеров: управление продуктом, управление программой, разработка, тестирование, удовлетворение потребителя, управление выпуском и архитектура. На примере системы учёта заявок показано, как распределение ответственности, итеративная разработка, управление рисками и подготовка внедрения помогают организовать работу команды. Таким образом, MSF представляет собой комплексный подход к созданию и внедрению программных решений, в котором техническое качество рассматривается вместе с бизнес-ценностью, удобством использования и готовностью системы к эксплуатации.